

Overall Detection

3.1 Introduction

MediaPipe is a data stream processing machine learning application development framework developed and open-sourced by Google. It is a graph-based data processing pipeline used to build applications that use multiple forms of data sources, such as video, audio, sensor data, and any time series data. MediaPipe is cross-platform and can run on embedded platforms (Raspberry Pi and similar), mobile devices (iOS and Android), workstations and servers, with support for mobile GPU acceleration. MediaPipe provides cross-platform, customizable ML solutions for real-time and streaming media.

The core framework of MediaPipe is implemented in C++ and provides support for Java and Objective C. The main concepts of MediaPipe include Packet, Stream, Calculator, Graph, and Subgraph.

MediaPipe features:

- End-to-end acceleration: Built-in fast ML inference and processing accelerate even on ordinary hardware.
- Build once, deploy anywhere: Unified solutions work on Android, iOS, desktop/cloud, web, and IoT.
- Ready-to-use solutions: Cutting-edge ML solutions that showcase all framework features.
- Free and open source: Framework and solutions under Apache 2.0, fully extensible and customizable.

3.2 MediaPipe Hands

Refer to section 1.2 Hand Detection. No further explanation needed here.

3.3 MediaPipe Pose

Refer to section 2.2 Posture Detection. No further explanation needed here.

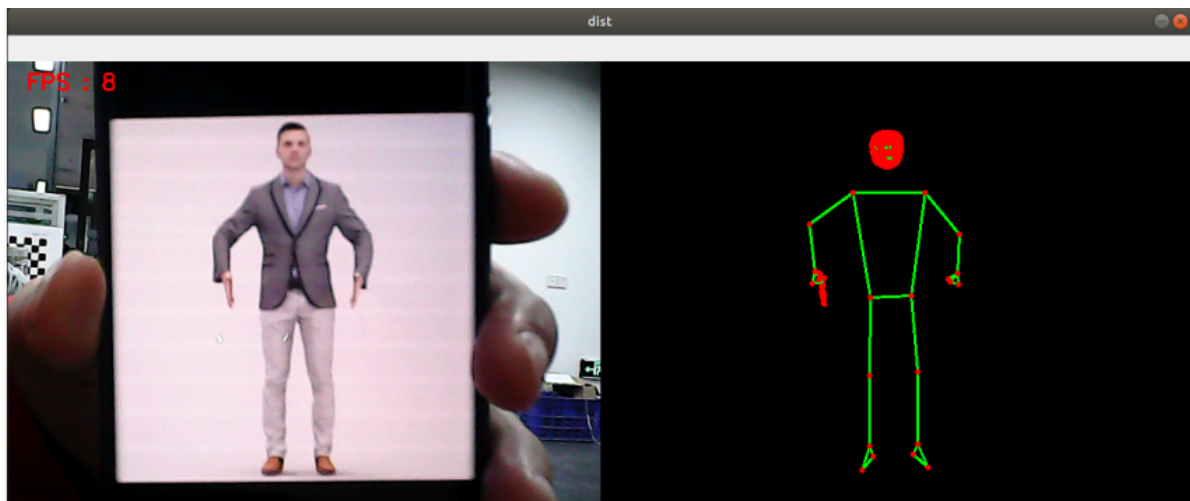
3.4 Overall Detection

Combining the content from the previous two sections, this section's example implements functionality that can detect both palms and human body.

3.4.1 Launch

Enter in terminal:

```
ros2 run dofbot_mediapipe 03_Holistic
```



3.4.2 Source Code

Source location:

/home/yahboom/dofbot_ws/src/dofbot_mediapipe/dofbot_mediapipe/03_Holistic.py

```
#!/usr/bin/env python3
# encoding: utf-8
import time
import rclpy
from rclpy.node import Node
import cv2 as cv
import numpy as np
import mediapipe as mp
from geometry_msgs.msg import Point
from dofbot_msgs.msg import PointArray
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError

class Holistic(Node):
    def __init__(self, staticMode=False, landmarks=True, detectionCon=0.5,
trackingCon=0.5):
        super().__init__('holistic_detector')
        self.publisher_ = self.create_publisher(PointArray, '/mediapipe/points',
10)

        self.bridge = CvBridge()

        self.mpHolistic = mp.solutions.holistic
        self.mpFaceMesh = mp.solutions.face_mesh
        self.mpHands = mp.solutions.hands
        self.mpPose = mp.solutions.pose
        self.mpDraw = mp.solutions.drawing_utils
        self.mpholistic = self.mpHolistic.Holistic(
            static_image_mode=staticMode,
            smooth_landmarks=landmarks,
            min_detection_confidence=detectionCon,
            min_tracking_confidence=trackingCon)

        self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0,
255), thickness=-1, circle_radius=3)
        self.drawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 255,
0), thickness=2, circle_radius=2)

        self.capture = cv.VideoCapture(0, cv.CAP_V4L2)
```

```

self.capture.set(6, cv.VideoWriter_fourcc('M', 'J', 'P', 'G'))
self.capture.set(cv.CAP_PROP_FRAME_WIDTH, 640)
self.capture.set(cv.CAP_PROP_FRAME_HEIGHT, 480)

if not self.capture.isOpened():
    self.get_logger().error("Failed to open the camera")
    return

self.get_logger().info(f"Camera FPS:
{self.capture.get(cv.CAP_PROP_FPS)}")
self.pTime = time.time()

self.timer = self.create_timer(0.03, self.process_frame)

def process_frame(self):
    ret, frame = self.capture.read()
    if not ret:
        self.get_logger().error("Failed to read frame")
        return

    frame, img = self.findHolistic(frame, draw=True)

    cTime = time.time()
    fps = 1 / (cTime - self.pTime)
    self.pTime = cTime
    text = "FPS : " + str(int(fps))
    cv.putText(frame, text, (20, 30), cv.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0,
255), 2)

    combined_frame = self.frame_combine(frame, img)
    cv.imshow('HolisticDetector', combined_frame)

    if cv.waitKey(1) & 0xFF == ord('q'):
        self.get_logger().info("Exiting program")
        self.capture.release()
        cv.destroyAllWindows()
        self.destroy_node()
        rclpy.shutdown()
        exit(0)

def findHolistic(self, frame, draw=True):
    pointArray = PointArray()
    img = np.zeros(frame.shape, np.uint8)
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    self.results = self.mpholistic.process(img_RGB)

    if self.results.face_landmarks:
        try:
            if draw: self.mpDraw.draw_landmarks(frame,
self.results.face_landmarks, self.mpFaceMesh.FACE_CONNECTIONS, self.lmDrawSpec,
self.drawSpec)
            self.mpDraw.draw_landmarks(img, self.results.face_landmarks,
self.mpFaceMesh.FACE_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
        except:
            if draw: self.mpDraw.draw_landmarks(frame,
self.results.face_landmarks, self.mpFaceMesh.FACEMESH_CONTOURS, self.lmDrawSpec,
self.drawSpec)

```

```

        self.mpDraw.draw_landmarks(img, self.results.face_landmarks,
self.mpFaceMesh.FACEMESH_CONTOURS, self.lmDrawSpec, self.drawSpec)

        for id, lm in enumerate(self.results.face_landmarks.landmark):
            point = Point()
            point.x, point.y, point.z = lm.x, lm.y, lm.z
            pointArray.points.append(point)
        if self.results.pose_landmarks:
            if draw: self.mpDraw.draw_landmarks(frame,
self.results.pose_landmarks, self.mpPose.POSE_CONNECTIONS, self.lmDrawSpec,
self.drawSpec)
            self.mpDraw.draw_landmarks(img, self.results.pose_landmarks,
self.mpPose.POSE_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
            for id, lm in enumerate(self.results.pose_landmarks.landmark):
                point = Point()
                point.x, point.y, point.z = lm.x, lm.y, lm.z
                pointArray.points.append(point)
        if self.results.left_hand_landmarks:
            if draw: self.mpDraw.draw_landmarks(frame,
self.results.left_hand_landmarks, self.mpHands.HAND_CONNECTIONS,
self.lmDrawSpec, self.drawSpec)
            self.mpDraw.draw_landmarks(img, self.results.left_hand_landmarks,
self.mpHands.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
            for id, lm in enumerate(self.results.left_hand_landmarks.landmark):
                point = Point()
                point.x, point.y, point.z = lm.x, lm.y, lm.z
                pointArray.points.append(point)
        if self.results.right_hand_landmarks:
            if draw: self.mpDraw.draw_landmarks(frame,
self.results.right_hand_landmarks, self.mpHands.HAND_CONNECTIONS,
self.lmDrawSpec, self.drawSpec)
            self.mpDraw.draw_landmarks(img, self.results.right_hand_landmarks,
self.mpHands.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
            for id, lm in enumerate(self.results.right_hand_landmarks.landmark):
                point = Point()
                point.x, point.y, point.z = lm.x, lm.y, lm.z
                pointArray.points.append(point)
        self.publisher_.publish(pointArray)
        return frame, img

def frame_combine(self, frame, src):
    if len(frame.shape) == 3:
        frameH, frameW = frame.shape[:2]
        srcH, srcW = src.shape[:2]
        dst = np.zeros((max(frameH, srcH), frameW + srcW, 3), np.uint8)
        dst[:, :frameW] = frame[:, :]
        dst[:, frameW:] = src[:, :]
    else:
        src = cv.cvtColor(src, cv.COLOR_BGR2GRAY)
        frameH, frameW = frame.shape[:2]
        imgH, imgW = src.shape[:2]
        dst = np.zeros((frameH, frameW + imgW), np.uint8)
        dst[:, :frameW] = frame[:, :]
        dst[:, frameW:] = src[:, :]
    return dst

def main(args=None):
    rclpy.init(args=args)

```

```
node = Holistic()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    pass
finally:
    node.capture.release()
    cv.destroyAllWindows()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```